game.py
237 lines
PY

README.md
92 lines
MD

I'm working on my Lab 4 Vibe Coding assignment. I have cloned the Galaga repository and recorded the original 10-second gameplay video. Now I want to fix the bugs and add the required features without messing up the existing game. look at the `bezier()` function in `game.py`. Please help me understand what's wrong with the formula and show me exactly what needs to be changed in both the x and y calculations. don't rewrite the entire file or change any other part of the game. Explain the fix in simple terms and tell me how to check whether it works.

Working through the mathematics of cubic Bézier curves.

What's wrong

A cubic Bézier curve blends four points using weights that always add up to 1:

```
B(t) = (1-t)³·P0 + 3(1-t)²t·P1 + 3(1-t)t²·P2 + t³·P3
```

Your code uses `u = 1 - t`, so the weights should be `u³`, `3u²t`, `3ut²` and `t³`.
Comparing them with your code:

Term	Correct	Your code	Status
P0	`u³`	`u ** 3`	✅
P1	`3u²t`	`3 * u * u * t`	✅
P2	`3ut²`	`3 * u * t`	❌ missing one `t`
P3	`t³`	`t ** 3`	✅

The README hints at checking both middle terms, but only the **third term (P2)** is wrong. It has `t¹` instead of `t²`, in both the x and y lines.

Why endpoints still look right: at `t=0` the P2 term is 0 because of `t`, and at `t=1` it is 0 because `u = 0`. So the curve starts and ends in the right place. In between, the weights add up to more than 1. At `t=0.5` they total `0.125 + 0.375 + 0.75 + 0.125 = 1.375`, so the curve gets pulled too far toward P2 and overshoots.

The exact change

Only two lines change. Everything else in the function stays the same.

x line

```python
# before
x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * p2[0] + t ** 3 * p3[0]
# after
x = u ** 3 * p0[0] + 3 * u * u * t * p1[0] + 3 * u * t * t * p2[0] + t ** 3 * p3[0]
```

y line

```python
# before
y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * p2[1] + t ** 3 * p3[1]
# after
y = u ** 3 * p0[1] + 3 * u * u * t * p1[1] + 3 * u * t * t * p2[1] + t ** 3 * p3[1]
```

The only difference is `3 * u * t * p2[...]` becoming `3 * u * t * t * p2[...]`.

How to check it works

1. Quick numeric test. Save this as `test_bezier.py` next to `game.py` and run `python test_bezier.py`:

```python
from game import bezier

# Test points chosen so the correct midpoint is easy to compute by hand
p0, p1, p2, p3 = (0, 0), (0, 100), (100, 100), (100, 0)

print("t=0  :", bezier(p0, p1, p2, p3, 0))      # expect (0, 0)
print("t=1  :", bezier(p0, p1, p2, p3, 1))      # expect (100, 0)
print("t=0.5:", bezier(p0, p1, p2, p3, 0.5))    # expect (50, 75)

# Weights should always sum to 1
for t in (0.1, 0.25, 0.5, 0.75, 0.9):
    u = 1 - t
    assert abs(u**3 + 3*u*u*t + 3*u*t*t + t**3 - 1) < 1e-9
print("weights sum to 1 ✓")
```

At `t=0.5`:

- Buggy code gives about `(87.5, 112.5)`. The y value of 112.5 is above both control points' y of 100, which a correct Bézier can never do.
- Fixed code gives `(50, 75)`.

2. Visual check. Run `python game.py` and watch the enemy entry flights and dives. The paths should now curve smoothly toward the control points without the weird mid-flight bulge or overshoot. Compare against your "before" video, since this is the difference to show in your "after" clip.

If you want, send me your results from the test and I can help with the next task (`enemy_tint`, `on_wave_start`, or `shield_charges`) without touching the rest of the file.

> The Bézier fix is done and I tested it successfully. look at the existing `enemy_tint(kind)` function and the code that calls it in `game.py`. I need to implement the enemy-colour feature described in the README, where the colours can vary as the waves progress. explain how to implement this with minimal changes while keeping the existing default colours as a fallback. also consider how the function can know the current wave, since it only receives `kind` as an argument. Show me the exact code changes and explain where each change should go. don't rewrite the whole file or modify the wave banner or shield feature yet. I want to apply and test this feature before moving on.

Adding wave context to enemy tint logic.

How the function can know the wave

`enemy_tint(kind)` only receives `kind`, so there are two options:

1. **Pass the wave in as a second argument.** `draw()` already has `self.wave`, so the call site just forwards it.
2. **Use a module-level global** like `CURRENT_WAVE`. This means extra bookkeeping somewhere else, and the natural place to set it is `spawn_wave` / `on_wave_start`, which you said not to touch yet.

Option 1 is the smaller change, so use it. Give the new parameter a default (`wave=1`) so any code still calling `enemy_tint(kind)` keeps working.

Change 1: replace the `enemy_tint` stub

It's near the top of the file. Keep it where it is and replace the stub (the docstring and `pass`) with this:

python

```
WAVE_PALETTES = [
    None,  # wave 1: use the default ENEMY_COLORS
    {"boss": (255, 200, 60), "red": (255, 140, 60), "blue": (60, 220, 200)},   # wave
    {"boss": (255, 90, 200), "red": (200, 60, 255), "blue": (100, 100, 255)},  # wave
    {"boss": (240, 240, 240), "red": (255, 220, 80), "blue": (120, 255, 120)}, # wave
]


def enemy_tint(kind, wave=1):
    """Return an (r, g, b) colour override for an enemy kind, or None for the default.
    palette = WAVE_PALETTES[(wave - 1) % len(WAVE_PALETTES)]
    if palette is None:
        return None
    return palette.get(kind)
```

- Wave 1 maps to `None`, so the original colours show.
- Waves 2 to 4 each get their own palette, and the list cycles (wave 5 reuses the wave 1 look, and so on), so it works for endless waves.
- `palette.get(kind)` returns `None` for an unknown kind, which also falls back to the default.

Change 2: pass the wave at the call site

In `Game.draw`, change one line:

```python
# before
color = enemy_tint(enemy.kind) or ENEMY_COLORS[enemy.kind]
# after
color = enemy_tint(enemy.kind, self.wave) or ENEMY_COLORS[enemy.kind]
```

The `or ENEMY_COLORS[...]` part stays, so the default colours remain the fallback. A non-empty tuple is truthy, so the override is used whenever one is returned.

Nothing else changes. `on_wave_start`, `shield_charges` and the rest of the game are untouched.

One thing to expect

The boss turns purple `(170, 90, 220)` after its first hit. That line comes after the tint in `draw()`, so it still overrides the wave colour, which is a useful damage cue. If you'd rather keep the wave colour on damaged bosses, you'd have to change that line, but I'd leave it for now.

How to test

1. Quick function check. Save as `test_tint.py` next to `game.py` and run it:

```python
from game import enemy_tint

print(enemy_tint("red"))        # None (wave 1 default)
print(enemy_tint("red", 1))     # None
print(enemy_tint("red", 2))     # (255, 140, 60)
print(enemy_tint("blue", 3))    # (100, 100, 255)
print(enemy_tint("boss", 5))    # None (cycles back to wave 1)
print(enemy_tint("ghost", 2))   # None (unknown kind falls back)
```

2. Visual check in the game. Clearing a wave takes a while, so for testing only, temporarily change this line in `reset()`:

```python
self.score, self.wave, self.time = 0, 2, 0.0   # was 1; revert after testing
```

Run `python game.py`. The enemies should appear in the wave 2 colours (orange, teal and yellow). Try 3 and 4 as well. Then set it back to `1` and confirm wave 1 looks like your original video. Play through a full wave at least once to see the colours change on their own.

Tell me how it goes, and we can move on to `on_wave_start` or `shield_charges`.

> The enemy tint feature is working now. look at how `on_wave_start(wave)` is currently called in `game.py` and how the game renders its HUD. I want to display a temporary banner like "WAVE 2" whenever a new wave starts. explain how to connect the existing hook to the rendering code with minimal changes. Show me exactly what to add or modify and where each change should go. Make sure the banner disappears after a short time and doesn't interfere with movement, shooting, scoring, or the existing HUD.

Designing font caching and banner placement for the display. ›

How the pieces connect

`on_wave_start(wave)` is called from `spawn_wave()`, which is a module-level function with no access to the `Game` object. So the hook can't draw anything or set `self.something`. The smallest bridge is a small module-level dict that the hook writes to and that `Game` reads from:

- `on_wave_start` sets the banner text and a timer.
- `Game.update` counts the timer down using `dt`.
- `Game.draw` renders the text while the timer is above 0.

The dict is mutated rather than reassigned, so no `global` statements are needed. Because `spawn_wave` is also called from `reset()`, the banner shows "WAVE 1" at game start and again after pressing `R`, with no extra code.

Change 1: replace the `on_wave_start` stub

Replace the docstring and `pass` of `on_wave_start`, and add the `BANNER` dict just above it:

python

```python
BANNER = {"text": "", "time": 0.0, "font": None}
BANNER_DURATION = 2.0


def on_wave_start(wave):
    """Called at the start of every wave; add banners, speed-ups, or palette swaps her
    BANNER["text"] = f"WAVE {wave}"
    BANNER["time"] = BANNER_DURATION
```

The `"font"` entry starts as `None` because fonts can't be created until `pygame.init()` has run. The draw code below creates it on first use.

Change 2: count the timer down in `Game.update`

Add one line right after `self.time += dt`:

```python
        self.time += dt
        BANNER["time"] = max(0.0, BANNER["time"] - dt)   # <-- add this
        self.cooldown -= dt
```

It sits below the `if self.state != "play": return` guard, so the timer only runs while the game is being played. It uses `dt` like everything else, so it doesn't depend on frame rate.

Change 3: draw the banner at the end of `Game.draw`

Add this block as the last lines of `draw()`, after the `if self.state == "lose":` block:

```python
        if BANNER["time"] > 0 and self.state == "play":
            if BANNER["font"] is None:
                BANNER["font"] = pygame.font.Font(None, 72)
            label = BANNER["font"].render(BANNER["text"], True, (255, 255, 120))
            screen.blit(label, label.get_rect(center=(WIDTH // 2, HEIGHT // 3)))
```

- Drawing it last puts it on top of everything, but it's purely visual and never touches game state.
- It is centered at about y=233, below the HUD line at the top, so the score, lives and wave text stay readable.
- The `self.state == "play"` check stops a frozen banner from showing next to "GAME OVER" if you die within the first 2 seconds of a wave.
- The 72-size font is created once and reused, not rebuilt every frame.

Why this doesn't interfere with anything

Movement, shooting, scoring, collisions and the HUD code are not modified. The banner only reads `BANNER` and draws text, and the only thing `update` does with it is decrement a number.

How to test

1. **Start:** run `python game.py`. "WAVE 1" should appear mid-screen and vanish after about 2 seconds.
2. **Interaction:** while it's visible, move, fire and shoot an enemy. Everything should respond normally, and the HUD at the top should be unchanged.
3. **Reset:** press `R` and the banner should appear again.

4. **Later waves:** temporarily set `self.wave` to `2` in `reset()` (revert afterwards) and run the game. You should see "WAVE 2" and the wave 2 colours from the previous step. Also play through a full wave once to confirm the banner triggers by itself when the last enemy dies.
5. **Game over:** die and confirm only "GAME OVER - Press R" shows.

If the banner stays too long or too short, change `BANNER_DURATION`. Tell me how it goes and we can move on to `shield_charges`.

The wave-start banner is working correctly. implement the final required feature now: `shield_charges(wave)`. inspect the existing function, the code that initialises the player's shield, and the `hit_player()` logic in `game.py`. I want one shield charge to be granted every three waves, as described in the README. make sure the feature uses the existing shield consumption and temporary invulnerability logic instead of creating a separate shield system. Show me the exact minimal code change and explain how it works. don't rewrite the game or modify the three features we've already completed.
Also tell me how to test the function and verify in the game that a shield charge is granted and consumed correctly

Checking shield integration with hits and HUD visibility. ›

The change

Only the `shield_charges` stub changes. Replace its docstring and `pass`:

```python
def shield_charges(wave):
    """Return how many hits the player's shield can absorb this wave, or None to disab
    return 1 if wave % 3 == 0 else None
```

Waves 3, 6, 9 and so on get one charge. Every other wave returns `None`, which disables the shield.

How it works with the existing code

Nothing else needs editing, because both call sites already use the function:

- **Game start and `R`:** `reset()` runs `self.shield = shield_charges(self.wave) or 0`. Wave 1 gives `None`, which becomes `0`, so there is no shield.
- **New wave:** `update()` runs the same line right after `spawn_wave`. When wave 3 begins, `self.shield` becomes `1`.
- **Consumption:** `hit_player()` already handles it. If `self.shield > 0`, it subtracts one, sets `invulnerable = 1.0`, and returns before any ship or life is lost. Once it hits 0, the next hit takes a ship or a life as usual.

This is the existing shield system with no new state. It also covers all damage sources, since enemy shots and ramming dives both go through `hit_player()`.

The call sites assign the shield at each wave start, so an unused charge doesn't carry over. Wave 3's charge is gone when wave 4 starts, and wave 6 gets a fresh one. This matches "one charge every 3 waves" and needs no extra code.

Test the function

Save as `test_shield.py` next to `game.py`:

```python
from game import shield_charges

for w in range(1, 10):
    print(w, shield_charges(w))
```

Expected output: `None` for waves 1, 2, 4, 5, 7, 8, and `1` for waves 3, 6, 9.

Verify in the game

The game has no shield indicator in the HUD, so you check it by watching the lives count and the ship's blinking.

1. In `reset()`, temporarily change `self.wave` from `1` to `3`. Revert it afterwards.
2. Run `python game.py` and let an enemy shot or diving enemy hit you once.
   - **Lives stay at 3**, and the ship blinks for about 1 second. That is the shield being consumed.
3. Get hit again.
   - **Lives drop to 2**, and the ship blinks for about 2 seconds. That is a normal hit, so the shield is gone.
4. Change `self.wave` to `1` and confirm the first hit costs a life immediately, with no shield.
5. Change it to `2` and then play through to wave 3 naturally. Your first hit in wave 3 should be absorbed, the first hit in wave 4 should not, and wave 6 should grant a new one. This step takes longer, so you can skip it if the temporary test looks right.

For a clearer check, you can temporarily add `print("shield left:", self.shield)` inside `hit_player()`, just before the `if self.shield > 0:` line. You should see `1` on the first hit and `0` after. Remove it when you're done.

Make sure `self.wave` is back to `1` before you record the "after" video. Because the shield first appears in wave 3, you'll need to reach wave 3 to show it. Starting at wave 3 for the clip works too, if you say so in the video.